Labo 08 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — Trois images pour trois environnements

Réponse. L'image testée en recette (`-staging`) n'est **pas** celle déployée en production (`-prod`) : deux builds, deux contenus, et la configuration de production n'a été exercée nulle part avant la mise en production. Il fallait **une seule image** `api:1.4.2`, construite une fois, et la configuration injectée au déploiement (`--env-file`, fichier monté, secrets). On perd : la garantie « ce qui a été testé est ce qui tourne », la traçabilité par digest, la possibilité de promouvoir la même image, et on gagne trois builds, trois scans, trois stockages.

Pourquoi. Une image est un artefact immuable identifié par son digest ; c'est ce digest que l'on promeut. Dès que la configuration est dedans, l'artefact change avec l'environnement.

Nuance. Un `ENV` de **valeur par défaut** dans l'image (`APP_PROFILE=default`, `SERVER_PORT=8080`) est légitime : il décrit l'image, il ne décrit pas un environnement.

Exemple.

```
podman run -d --name api-staging --env-file staging.env api:1.4.2
podman run -d --name api-prod    --env-file prod.env    api:1.4.2     # même image, même digest
```

Question 2 — Priorité des sources

Réponse. `staging`, `prod`, `prod`, `shell`. L'ordre des options n'a pas d'importance parce que Podman n'applique pas les sources « dans l'ordre de la ligne » mais par **catégorie** : il charge d'abord l'`ENV` de l'image, puis les fichiers `--env-file`, puis les `-e`, les derniers écrasant les premiers. `-e APP_PROFILE` sans valeur copie la variable du shell de l'hôte.

Pourquoi. C'est une règle de précédence, pas une séquence : la valeur la plus « proche » de la commande gagne (ligne de commande > fichier > image).

Nuance. Dans un `--env-file`, la **dernière** ligne pour une même clé gagne ; et Docker a la même règle `-e` > `--env-file`.

Exemple.

```
podman run --rm -e APP_PROFILE=prod --env-file api.env api:env env | grep APP_PROFILE   # prod
```

Question 3 — Port 9090 et `-p 18080:8080`

Réponse. `SERVER_PORT=9090` a fait écouter l'application sur **9090** ; `-p 18080:8080` relaie vers 8080, où plus personne n'écoute : connexion refusée dans le conteneur, `curl` sans réponse. `EXPOSE 8080` n'y change rien : il décrit l'image, pas la configuration réelle du processus. Il faut aligner : `-p 18080:9090`, ou ne pas changer le port.

Pourquoi. Trois couches indépendantes : l'application écoute où sa configuration le dit, `-p` relaie où on le lui dit, `EXPOSE` documente. Aucune ne vérifie les deux autres.

Nuance. Dans un réseau user-defined (labo 07), les autres conteneurs devraient aussi appeler `api:9090`. Changer un port par variable a un rayon d'action plus large qu'il n'y paraît.

Exemple.

```
podman run -d --name api2 -p 18080:9090 -e SERVER_PORT=9090 api:1.0 && sleep 2 && curl -s localhost:18080/actuator/health
```

Question 4 — Mot de passe en argument

Réponse. Du plus au moins grave : (1) **les logs** — l'application affiche ses arguments (`Arguments recus : --db-password=Secr3t`), et les logs sont copiés, centralisés, conservés ; (2) **le `ps` de l'hôte** — tout utilisateur de la machine lit `ps -eo args` ; (3) **`podman inspect`** (`Config.Cmd`) — toute personne ayant accès au moteur ; (4) **`podman top api args`** — idem. S'y ajoute l'historique du shell (`~/.bash_history`).

Pourquoi. La ligne de commande d'un processus est une donnée publique du système (`/proc/<pid>/cmdline`), et la commande d'un conteneur est stockée dans sa configuration.

Nuance. Même sans affichage par l'application, le `ps` suffit. Un argument n'est jamais un canal pour un secret, dans ou hors conteneur.

Exemple.

```
podman logs api | head -1 ; podman inspect api --format '{{json .Config.Cmd}}' ; ps -eo args | grep -c 'db-password=Secr3t'
```

Question 5 — Mot de passe en variable

Réponse. Oui, c'est mieux : il disparaît des logs d'arguments, de `ps` et de `Config.Cmd`. Il reste visible dans `podman inspect` (`Config.Env`), `podman exec api env`, `/proc/1/environ` dans le conteneur, et dans tout rapport d'erreur qui vide l'environnement. Avec `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`, `Config.Env` ne contient plus que ce **chemin** : la valeur n'est lisible que par un processus du conteneur qui ouvre le fichier.

Pourquoi. L'environnement est une métadonnée du conteneur, stockée par le moteur ; un fichier monté n'en est pas une.

Nuance. `podman exec db cat /run/secrets/db_password` reste possible pour qui a accès au moteur. Le gain est d'enlever la valeur des **métadonnées** et des exports (`inspect`, Compose, tickets), pas de la rendre inaccessible à l'administrateur.

Exemple.

```
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES   # POSTGRES_PASSWORD_FILE=/run/secrets/db_password
```

Question 6 — Spring Boot et l'environnement

Réponse. Le *relaxed binding* convertit un nom de variable en nom de propriété : majuscules → minuscules, `_` → `.` (`SPRING_DATASOURCE_URL` → `spring.datasource.url`). `SPRING_PROFILES_ACTIVE=prod` active le profil `prod` : Spring Boot charge en plus `application-prod.yaml` et les beans annotés `@Profile("prod")`. En cas de conflit, la **variable**

d'environnement gagne sur `/config/application.yaml` (fichier externe, hors du JAR), qui gagne sur `application.yaml` dans le JAR.

Pourquoi. Spring Boot ordonne ses `PropertySources` du plus externe au plus interne, précisément pour permettre l'injection sans rebuild.

Nuance. Les arguments `--spring.datasource.url=…` passent devant les variables — mais ce sont des arguments, donc visibles (question 4). Et `spring.config.import=optional:configtree:/run/secrets/` fait de chaque fichier de `/run/secrets/` une propriété.

Exemple.

```
podman run -d --name api -e SPRING_PROFILES_ACTIVE=prod -e
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app -v
./application.yaml:/config/application.yaml:ro vraie-api:1.0
```

Question 7 — Dockerfile contestable

Réponse. `ENV SPRING_PROFILES_ACTIVE=prod` : fige un environnement dans l'image — à passer par `-e` au déploiement. `ENV SPRING_DATASOURCE_PASSWORD=Pr0d!` : un secret dans l'image, lisible par `image inspect` et `history` — à passer par secret (`_FILE` / `configtree`). `COPY .env /app/.env` : un fichier de configuration locale, probablement avec des secrets, dans une couche — à mettre dans `.dockerignore`, et à fournir au `run` par `--env-file` ou montage. Corrigé :

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Pourquoi. L'image ne doit contenir que ce qui est vrai partout : le code et ses défauts. Tout le reste est injecté.

Nuance. Un `ENV SPRING_PROFILES_ACTIVE=default` ou `ENV JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75` resterait acceptable : des défauts, pas des environnements.

Exemple.

```
podman image inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -i -E
'password|profile'   # doit être vide
```

Question 8 — `podman secret` en pratique

Réponse. `Config.Env` contient `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` et rien d'autre sur le mot de passe ; `ls -l /run/secrets/` montre `-r--r--r-- 1 root root 6 … db_password` ; `podman secret inspect db_password` donne l'ID, les dates, le pilote `file` et son chemin, **pas la valeur**. `--showsecret` l'affiche : preuve que le stockage est un rangement lisible par votre utilisateur, pas un chiffrement.

Pourquoi. Le secret est copié dans le conteneur au démarrage comme un fichier ; le moteur ne le met ni dans l'environnement ni dans la configuration du conteneur.

Nuance. Le fichier appartient à `root` et est lisible par tous dans le conteneur (`444`) : un processus non-root le lit. `--secret db_password,mode=0400,uid=70` resserre les droits.

Exemple.

```
podman secret inspect --showsecret db_password --format '{{.SecretData}}'   # S3cr3t
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'          # 0
```

Question 9 — « Autant mettre `-e` »

Réponse. Par rapport à `-e`, `podman secret` retire la valeur de `podman inspect`, de l'environnement du processus (en mode fichier), des fichiers Compose et des commandes dans l'historique du shell ; il la crée **une fois** et la référence par nom partout ; et il la monte avec des droits contrôlables. Ce qu'il n'apporte pas : chiffrement au repos, contrôle d'accès, journal des lectures, rotation. C'est exactement ce qu'ajoute un gestionnaire de secrets : qui a lu quoi et quand, chiffrement par clé maître, renouvellement automatique et jetons de courte durée.

Pourquoi. `podman secret` résout le problème de **l'exposition accidentelle** ; un gestionnaire résout celui de la **gouvernance**.

Nuance. Sur un poste de développement, `podman secret` est largement suffisant. Le pilote `pass` (GPG) existe pour chiffrer localement.

Exemple.

```
podman secret ls ; podman secret inspect db_password --format '{{.Spec.Driver.Name}}'   # file
```

Question 10 — Le build réussit sans secret

Réponse. Le second build a trouvé toutes ses instructions **en cache**, y compris le `RUN --mount=type=secret`, qui n'a donc pas été réexécuté — et un `RUN` non exécuté ne réclame pas de secret. Pour le voir échouer : `podman build --no-cache`, ou modifier une instruction antérieure. Conséquence : un jeton expiré ne se détecte pas tant que le cache tient ; sur un agent de CI avec cache persistant, le build « marche » jusqu'au jour où une dépendance change.

Pourquoi. Le cache est indexé sur l'instruction et le contexte, pas sur les secrets fournis : fournir ou non un secret ne l'invalide pas.

Nuance. C'est aussi une protection : le secret n'étant pas dans la clé de cache, sa valeur n'est pas dérivable du cache.

Exemple.

```
podman build --no-cache -t app .   # Error: … secret required but no secret with id "token" found
```

Question 11 — `ARG` contre `--mount=type=secret`

Réponse. Avec `ARG NPM_TOKEN`, le jeton est dans les métadonnées de la couche : `podman history --no-trunc` affiche `|1 NPM_TOKEN=… /bin/sh -c npm ci`. Avec le montage de secret, il n'est nulle part : `history` montre la ligne `--mount=type=secret,id=npm_token npm ci`, sans valeur, et `/run/secrets` n'existe pas dans l'image. Le multi-stage protège **partiellement** l'`ARG` : si l'`ARG`

est déclaré dans un stage jeté, l'image finale n'en a pas l'historique. Mais une `ARG` dans le stage final reste visible, et l'image du stage de build (`--target`) garde tout.

Pourquoi. `ARG` est substitué dans la commande enregistrée de la couche ; un montage n'est pas enregistré.

Nuance. Buildah et BuildKit traitent les deux de la même façon. La seule différence : Buildah ignore `# syntax=` et n'en a pas besoin.

Exemple.

```
podman history --no-trunc app --format '{{.CreatedBy}}' | grep -i token
```

Question 12 — `.env` dans Git

Réponse. (1) **Git n'oublie pas** : même supprimé au commit suivant, le fichier reste dans l'historique, dans tous les clones et toutes les forks. (2) **Le développement n'est pas isolé** : les mots de passe de dev ouvrent souvent des services partagés (base de test, registry, API tierce en mode bac à sable), et les scanners de fuites (Gitleaks, GitHub secret scanning) le signaleront. (3) **L'habitude** : le réflexe « c'est juste du dev » finit par commettre une valeur de recette. Pratique : `.env` dans `.gitignore` et `.dockerignore`, un `.env.example` commis avec les clés et des valeurs vides, et pour la CI, des secrets injectés par l'outil.

Pourquoi. Un dépôt est un support de diffusion ; tout ce qui y entre doit pouvoir être public.

Nuance. Un `.env` commis **sans aucun secret** (ports, noms de services) est acceptable, et courant avec Compose (labo 09). Le problème est le contenu, pas le fichier.

Exemple.

```
git log --all -p -- .env | grep -i password   # ce que l'historique garde
```

Question 13 — Fuite dans un ticket

Réponse. (1) Considérer le secret **compromis** : la question « qui l'a vu » n'a pas de réponse fiable. (2) **Faire tourner** le secret : nouveau mot de passe côté base, nouvelle valeur dans le gestionnaire de secrets ou `podman secret`, redéploiement des consommateurs. (3) **Révoquer** l'ancien, vérifier les journaux d'accès à la base pour la période. (4) Nettoyer le ticket, **ensuite**, et traiter la cause : pourquoi un `inspect` complet était-il nécessaire, pourquoi contenait-il un secret (donc `-e` au lieu d'un fichier). Supprimer le commentaire ne fait rien : il a été lu, notifié par courriel, peut-être indexé. La rapidité dépend de l'architecture : secret référencé par nom et injecté au démarrage → quelques minutes ; secret copié à la main dans des fichiers de configuration sur des serveurs → des heures et des oublis.

Pourquoi. La seule réponse sûre à une fuite est de rendre la valeur fuitée **inutile**.

Nuance. C'est l'argument décisif pour la rotation automatique : si changer un secret est une opération quotidienne et sans douleur, une fuite est un incident mineur.

Exemple.

```
podman secret rm db_password && printf 'nouveau' | podman secret create db_password - && podman restart api db
```